

ComfyUI Script-to-Video Suite

Official Users Manual

A structured automation pipeline that converts long-form PDF scripts into cinematic storyboard panels and optimized prompts for AI-based image and video generation.

1. Core Pipeline Overview

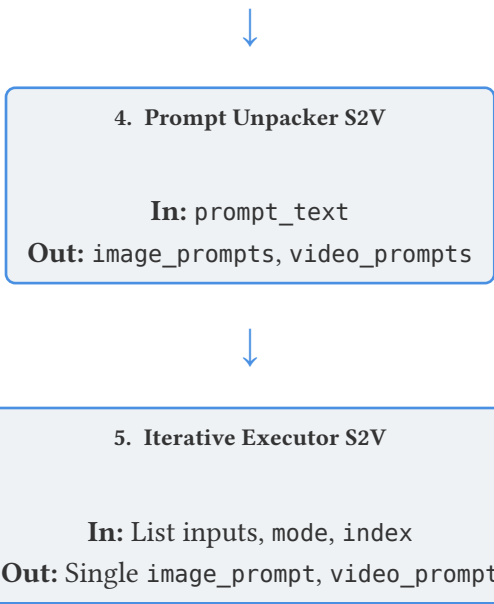
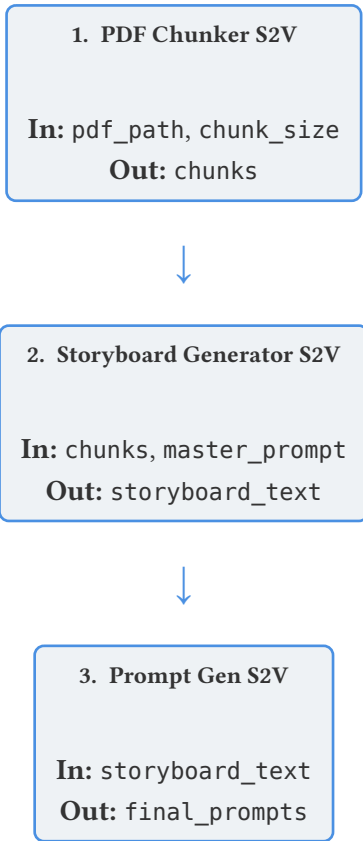
To generate a complete video sequence from a PDF script, the core nodes must be connected in the following specific execution order. Maintaining this sequence ensures correct dependency resolution and prevents downstream processing errors.

Order of Execution:

1. PDF Chunker (S2V)
2. Storyboard Generator (S2V)
3. Prompt Generator (S2V)
4. Prompt Unpacker (S2V)
5. Iterative Executor (S2V)

1.1. Pipeline Architecture Flow

The workflow acts as an automated funnel, translating documents into visual data. Below is the input/output relationship between the nodes:



2. Core Nodes Reference

2.1. PDF Chunker (S2V)

Purpose: Extracts and segments screenplay text into structured overlapping chunks.

Inputs:

- pdf_path – Path to source PDF.
- chunk_size – Character count per segment.
- overlap_size – Shared context between chunks.

Outputs:

- chunks – Ordered list of text segments.

Config: Recommended size is 2000-4000 characters with 10% overlap.

2.2. Storyboard Generator (S2V)

Purpose: Transforms text chunks into cinematic storyboard panels using Gemini AI.

Inputs:

- chunks – Output from the PDF Chunker.
- master_prompt – Directs the AI on style.

Outputs:

- storyboard_text – Detailed scene breakdown including camera directions and dialogue.

2.3. Prompt Generator (S2V)

Purpose: Converts storyboard panels into optimized prompts for models like WanVideo.

Inputs:

- storyboard_text

Outputs:

- final_prompts – Combined text block containing image prompts and video flow prompts.

2.4. Prompt Unpacker (S2V)

Purpose: Parses the aggregated text from the Prompt Generator into clean, iterable Python lists.

Inputs:

- prompt_text

Outputs:

- image_prompts (List)
- video_prompts (List)
- meta_summary (String Context)

2.5. Iterative Executor (S2V)

Purpose: Smartly sequences the generation of scenes to avoid GPU VRAM crashes.

Inputs:

- image_prompts & video_prompts
- mode (Manual / Auto)

Outputs:

- image_prompt & video_prompt (Single strings outputted one by one).

3. Enhancement Nodes (Optional)

These nodes can be injected into the main pipeline to improve visual consistency or automate post-processing.

3.1. RAG Consistency Engine

Purpose: Injects predefined character and location data to ensure cross-scene visual consistency.

Placement: Between Storyboard Gen and Prompt Gen.

Inputs: storyboard_text, enable_rag.

Outputs: enriched_storyboard.

3.2. Auto LoRA Loader (S2V)

Purpose: Dynamically scans an image prompt to identify a single main character trigger, then automatically loads the corresponding LoRA model from your loras/ folder by bypassing manual string-matching.

Inputs:

- image_prompt (STRING): The narrative text to analyze for character names.
- lora_strength (FLOAT): The impact strength of the selected LoRA (Default: 1.0).

Outputs:

- lora_stack (LORA_STACK): Output containing the configured LoRA to be passed to a model loader.

3.3. Multi LoRA Loader (S2V)

Purpose: Intended for use when multiple characters or stylistic elements are explicitly required. It resolves provided LoRA names to paths, validates them, and prepares a stack for merging.

Inputs:

- lora_stack (LORA_STACK): Accepts tuples of (name, strength).

Outputs:

- `loras_list` (WANVIDLORA): A list of properly configured LoRA dictionaries ready for downstream video model processing.

3.4. Fighting Scene Detector (S2V)

Purpose: Determines if a given prompt describes a combat, action, or fighting scene via a quick semantic analysis ping to the Gemini Relay Server.

Inputs:

- `input` (STRING): The prompt text to evaluate.

Outputs:

- `condition` (BOOLEAN): Returns True if action/combat is detected, False otherwise.

3.5. Dragon Ball LoRA Conditional (S2V)

Purpose: Acts as a conditional gatekeeper, typically receiving the boolean condition from the Fighting Scene Detector. If True, it loads high-action LoRA weights (e.g., Dragon Ball style); if False, it passes None.

Inputs:

- `condition` (BOOLEAN): The True/False trigger.
- `lora_name` (STRING): Selection from your `loras` folder.
- `strength` (FLOAT): Impact strength weight.

Outputs:

- `lora` (WANVIDLORA): The conditioned LoRA configuration, or None.

3.6. Video Merger (S2V)

Purpose: Combines all rendered .mp4 scene outputs into a single, cohesive master video file.

Placement: Final execution step after scenes are saved.

Inputs: `directory_path` (Output folder), `transition_duration`, `max_height`.

Outputs: `output_path` (Final MP4 file).

Behavior: Handles crossfade transitions, resolution normalization, and audio leveling safely.

—

4. System Infrastructure

4.1. Gemini Relay Client

Purpose: A foundational dependency for the AI generation nodes. It routes all language processing requests to an external, lightweight relay server, bypassing direct API blocking or geographical restrictions in ComfyUI.

Configuration Requirements: The Storyboard and Prompt Generators invisibly rely on this client. To function:

1. Run the external `gemini-relay` Python server on port 8080.
2. The relay server must be configured with your `GEMINI_API_KEY`.
3. If running remotely or exposed via ngrok, set `RELAY_SERVER_URL` in your `.env` file to point to your active relay instance.